Hand-Object Detector Manual Run

Purpose

This procedure runs the Shan hand-object detector on two test images and produces reference outputs for comparison with EgoModelKit.

Inputs:

- `hand_holding_ball.jpg`
- `hand_holding_cup.jpg`

Outputs for each image:

- `_det.png` — rendered detections
- `_shan.json` — structured predictions
- `_shan.pkl` — Python/NumPy predictions

The procedure starts from a clean x86-64 Linux account on newer Linux GPU machines.

Required files

Place the following files in `~/Downloads`:

hand_object_detector.zip
faster_rcnn_1_8_132028.pth
hand_holding_ball.jpg
hand_holding_cup.jpg

The supplied repository ZIP, which can be downloaded from the GitHub repository, uses commit: 70146ca Export JSON and pickle predictions

It is based on the original upstream Shan commit:
e6eec712a498ec7844b97893c8d012cea1a71e09

Commit `70146ca` only adds JSON and pickle export. It does not intentionally change model inference, thresholds, or rendered detections.

Deviations from the original Hand-Object-Detector README

This procedure intentionally differs from the original repository README in three areas:

1. **Structured outputs:** The original code only creates `_det.png`. Commit `70146ca` also creates `_shan.json` and `_shan.pkl` for comparison with EgoModelKit.
2. **CPU execution:** The README recommends PyTorch 1.12.1 with CUDA 11.3. This procedure uses the CPU-only PyTorch 1.12.1 environment because newer Linux GPU machines are not compatible with that legacy CUDA environment.
3. **Dependency versions:** Compatible NumPy, SciPy, and OpenCV versions are used so the repository can be installed on current Linux systems. OpenCV 5.0 is also needed to decode the cup input, which contains AVIF image data despite its `.jpg` filename.

Why the reference run uses CPU

The Shan detector supports both CPU and CUDA execution. CUDA is enabled only when `--cuda` is supplied. Without that option, the checkpoint, model, and detection calculations run on CPU.

CPU execution allows the original model and checkpoint to run without changing the detector's inference or post-processing logic. An NVIDIA GPU is therefore not required to produce expected results for these two images. GPU acceleration is mainly needed for practical performance on large image collections or videos and for the current CUDA-based EgoModelKit runtime.

The tested reference environment is x86-64 Linux. This procedure does not establish support for current Intel or Apple Silicon macOS systems because the legacy PyTorch dependencies and compiled C++ detector extension have not been validated there.

Because EgoModelKit may run with a newer GPU-based PyTorch environment, small floating-point differences are possible. Detection counts and categorical outputs must match exactly. Bounding boxes, scores, and offsets should be compared with a small numerical tolerance.

1. Verify system tools

Confirm that the required compiler and utilities are installed:

```
command -v git gcc g++ make wget unzip
```

Each command should print a path.

Install any missing tools when the account has administrator access:

```
sudo apt update
sudo apt install -y git build-essential wget unzip
```

2. Install Miniconda

Download and install Miniconda for the current Linux account:

```
cd ~/Downloads

wget -O Miniconda3.sh \
https://repo.anaconda.com/miniconda/Miniconda3-latest-Linux-x86_64.sh

bash Miniconda3.sh -b -p "$HOME/miniconda3"
```

Enable Conda in the current terminal:

```
eval "$("$HOME/miniconda3/bin/conda" shell.bash hook)"
```

Enable Conda automatically in future terminals:

```
conda init bash
```

Confirm the installation:

```
conda --version
```

3. Extract the repository

Create a workspace and extract the supplied repository:

```
mkdir -p ~/hoc-reference
cd ~/hoc-reference
unzip ~/Downloads/hand_object_detector.zip
cd hand_object_detector
```

Confirm the repository version:

```
git branch --show-current
git log -1 --oneline
```

Expected:

```
master
70146ca Export JSON and pickle predictions
```

If necessary, check out the required commit directly:

```
git checkout 70146ca
```

4. Create the Conda environment

Create and activate the environment:

```
conda create -y \
  --name handobj_new \
  python=3.8

conda activate handobj_new
```

Install CPU-only PyTorch 1.12.1:

```
conda install -y \
  pytorch==1.12.1 \
  torchvision==0.13.1 \
  torchaudio==0.12.1 \
  cpuonly \
  -c pytorch
```

Install the repository requirements and tested compatibility versions:

```
python -m pip install --upgrade pip
python -m pip install -r requirements.txt

python -m pip install \
  numpy==1.24.3 \
  scipy==1.10.1 \
  opencv-python==5.0.0.93
```

5. Compile the detector extension

From the repository root:

```
cd lib
python setup.py build develop
cd ..
```

Confirm that the environment and compiled extension load:

```
python - <<'PY'
import cv2
import torch
import torchvision

import _init_paths
from model.roi_layers import nms

print("PyTorch:", torch.__version__)
print("TorchVision:", torchvision.__version__)
print("OpenCV:", cv2.__version__)
print("CUDA available:", torch.cuda.is_available())
print("Detector extension import: PASS")
PY
```

Expected key output:

```
PyTorch: 1.12.1
TorchVision: 0.13.1
OpenCV: 5.0.0
CUDA available: False
Detector extension import: PASS
```

6. Place the checkpoint

Create the expected model directory and copy the checkpoint:

```
mkdir -p models/res101_handobj_100K/pascal_voc

cp ~/Downloads/faster_rcnn_1_8_132028.pth \
  models/res101_handobj_100K/pascal_voc/
```

Confirm the checkpoint:

```
sha256sum \
  models/res101_handobj_100K/pascal_voc/faster_rcnn_1_8_132028.pth
```

Expected SHA-256:

```
3444e3b992417cb6adfcd71539ca8c92602c21a3e2fdfff4628dbdbdf8a2dd03
```

7. Prepare the input images

Create a separate run directory:

```
RUN_ROOT="$HOME/hoc-reference-run"

mkdir -p \
  "$RUN_ROOT/images" \
  "$RUN_ROOT/results"
```

Copy the original images without resizing, converting, or re-encoding them:

```
cp ~/Downloads/hand_holding_ball.jpg \
  "$RUN_ROOT/images/"

cp ~/Downloads/hand_holding_cup.jpg \
  "$RUN_ROOT/images/"
```

Confirm that OpenCV can decode both files:

```
python - "$RUN_ROOT/images" <<'PY'
from pathlib import Path
import sys
import cv2

image_dir = Path(sys.argv[1])

for path in sorted(image_dir.iterdir()):
    image = cv2.imread(str(path))

    if image is None:
        raise RuntimeError(f"Could not decode {path}")
```

```
    print(path.name, list(image.shape), "PASS")
PY
```

Expected:

```
hand_holding_ball.jpg [360, 461, 3] PASS
hand_holding_cup.jpg [4029, 3000, 3] PASS
```

8. Run the detector

From the repository root:

```
cd ~/hoc-reference/hand_object_detector
conda activate handobj_new

Run both images:
python demo.py \
  --dataset pascal_voc \
  --cfg cfgs/res101.yml \
  --net res101 \
  --load_dir models \
  --image_dir "$RUN_ROOT/images" \
  --save_dir "$RUN_ROOT/results" \
  --checksession 1 \
  --checkepoch 8 \
  --checkpoint 132028 \
  --thresh_hand 0.5 \
  --thresh_obj 0.5
```

Do not add `--cuda`.

The original `demo.py` loop may process one image an additional time. With two inputs, three progress messages may appear. The additional pass overwrites the same output files and does not require another run.

9. Confirm the outputs

List the generated files:

```
find "$RUN_ROOT/results" \

  -maxdepth 1 \
  -type f \
  -printf '%f\n' \
  | sort
```

Expected:

```
hand_holding_ball_det.png
hand_holding_ball_shan.json
hand_holding_ball_shan.pkl
hand_holding_cup_det.png
hand_holding_cup_shan.json
hand_holding_cup_shan.pkl
```

The completed reference results are located at:
~/hoc-reference-run/results

10. Inspect the JSON outputs

Display the ball result:

```
python -m json.tool \
  "$RUN_ROOT/results/hand_holding_ball_shan.json"
```

Display the cup result:

```
python -m json.tool \
  "$RUN_ROOT/results/hand_holding_cup_shan.json"
```

Each JSON file contains:

```
image_name
hands
objects
thresholds
```

Each hand row contains:

```
x1, y1, x2, y2,
confidence,
contact_state,
offset_magnitude,
offset_dx,
offset_dy,
hand_side
```

Only the first five object values are meaningful:

```
x1, y1, x2, y2, confidence
```